Question 9 - Containerization and orchestration report

Question 9: Prepare a containerization and orchestration report describing the architecture, deployment strategy, scaling results, and observations with screenshots.

Executed on an Ubuntu 24.04.5 LTS OrbStack machine (`ensp461-lab2`, arm64). Source, logs and screenshots: https://github.com/ksamirdev/ensp461-lab2.

1. Objective

Package a small web service as a container image, run it alone and with a Redis dependency, deploy it to a Kubernetes cluster with zero-downtime updates, and automate the path from `git push` to a verified running version.

2. Tools and environment

Component	Version
Host	Ubuntu 24.04.5 LTS, OrbStack machine `ensp461-lab2`, arm64
Docker Engine / Compose / Buildx	29.9.3 / v5.7.0 / v0.38.1
Minikube / Kubernetes / kubectl	v1.39.0 (docker driver, containerd 2.3.4) / v1.37.0 / v1.37.2
Jenkins	2.580.2 LTS on OpenJDK 21.0.12, systemd service
Registry	Distribution `registry:2`, `localhost:5001` (host) and `lab2-registry:5000` (cluster)
Application	Python 3.12, Flask 3.1, gunicorn 23, Redis 7

3. Architecture

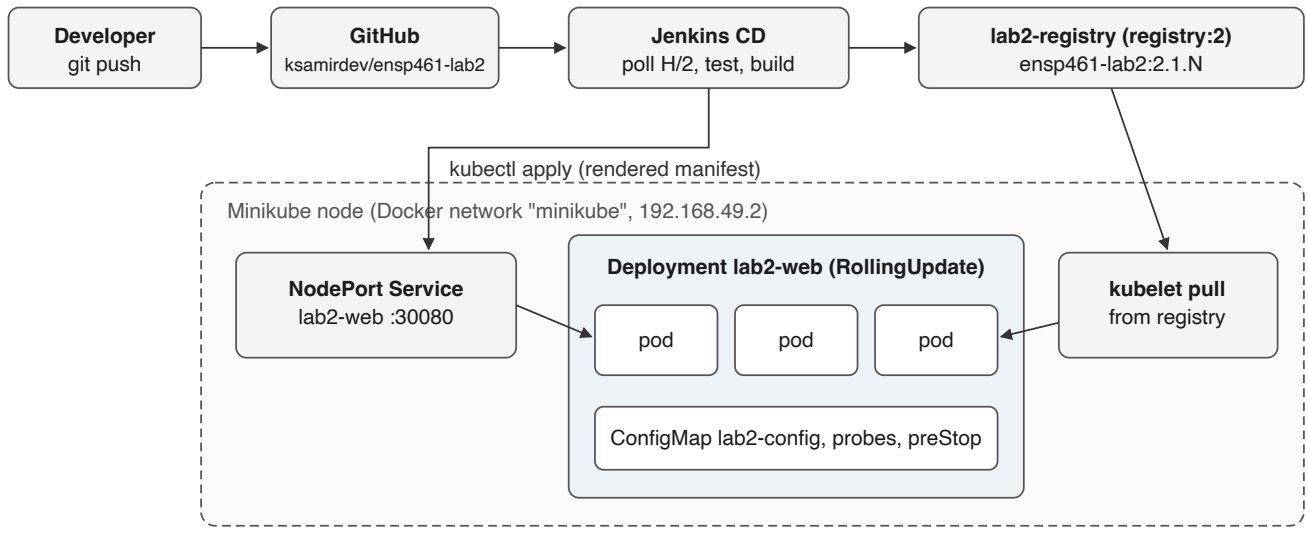


Fig: Containerization and orchestration architecture

A push to `main` is detected by Jenkins SCM polling. The pipeline runs the unit tests, builds `ensp461-lab2:2.1.<build>`, pushes it to the self-hosted registry, applies the manifests with that image, waits for the rollout and checks `/health` reports the new version. The kubelet on the Minikube node pulls the image from the same registry over the `minikube` Docker network. Users reach the pods through a NodePort Service that only routes to pods passing the readiness probe.

4. Containerization

- `docker/Dockerfile`: slim Python base, dependencies before source for caching, non-root user (uid 10001), `HEALTHCHECK`, version as a build argument.
- Image: 227 MB, 9 filesystem layers; the app's own layers add about 20 MB.
- `docker/docker-compose.yml`: web and Redis, health-gated start order, named volume for Redis data.
- Storage (Q3): writable layer lost with the container; named volume and bind mount persist.
- Networking (Q4): name resolution only on user-defined networks; separate networks are isolated.

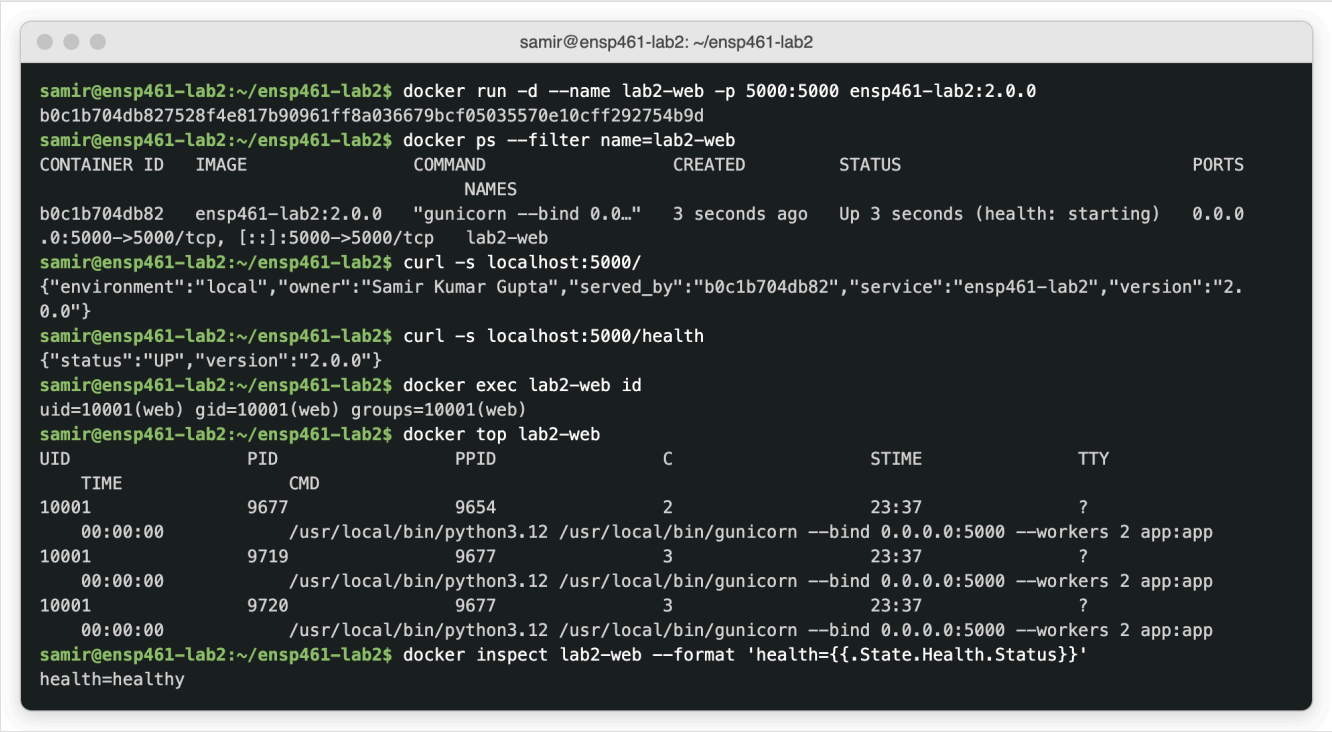


Fig: Container run: HTTP responses, non-root user, healthy status

5. Deployment strategy

`RollingUpdate` with `maxSurge: 1` and `maxUnavailable: 0`, readiness and liveness probes on `/health`, and a `preStop` sleep of 5 s.

Operation	Without preStop	With preStop
Rolling update 2.0.0 to 2.0.1	29 ok, 5 failed	61 ok, 0 failed
Rollback	80 ok, 2 failed	30 ok, 0 failed
Bad image (`9.9.9-missing`)		rollout stalled, old pods kept serving, no outage

`maxUnavailable: 0` alone kept capacity but not correctness: terminating pods stopped serving while still receiving traffic. Draining with `preStop` removed every failed request in the measurement. Rollback is `kubectl rollout undo`, which scales the previous ReplicaSet back up.

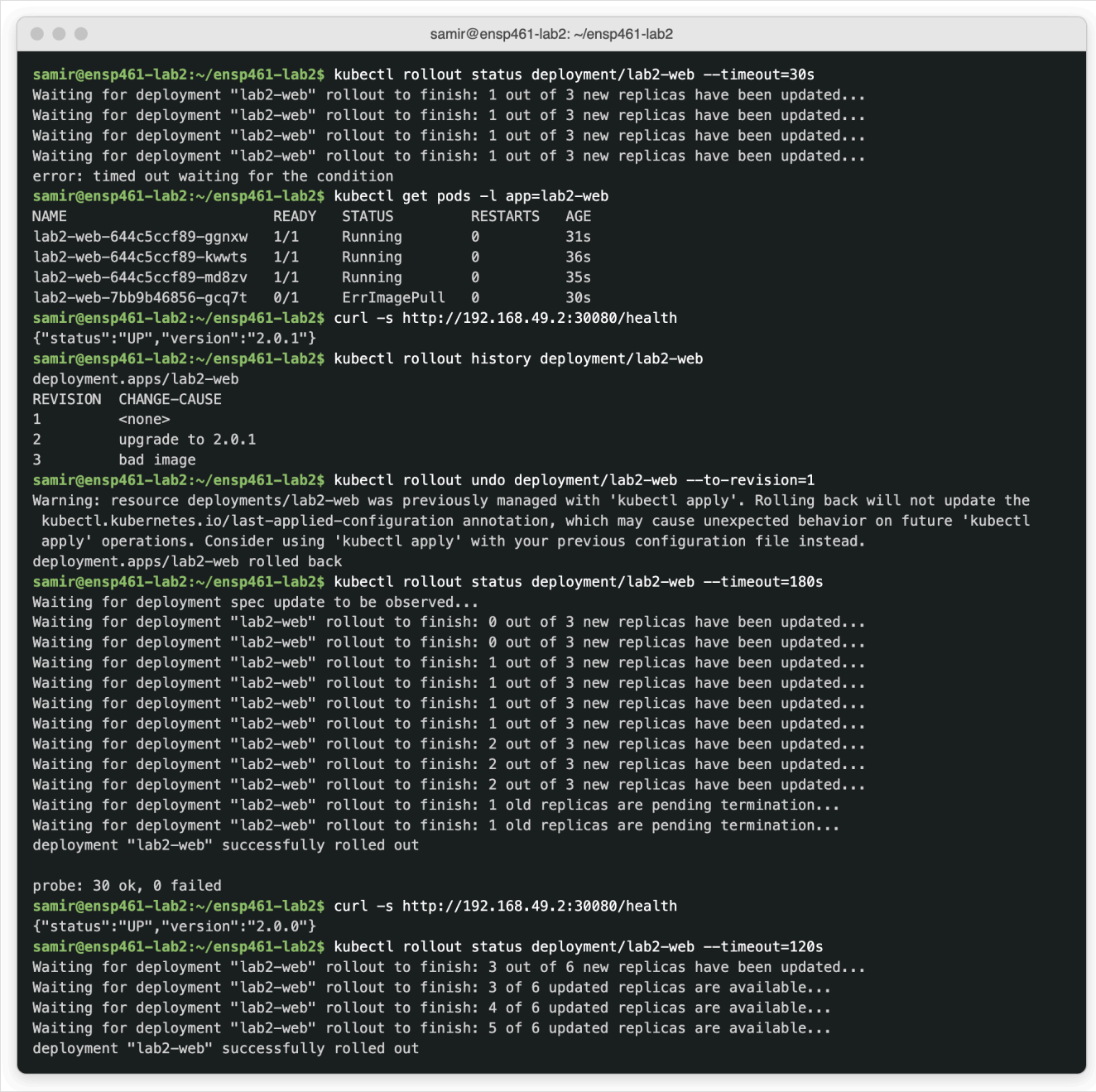


Fig: Bad image stalls the rollout; rollback to revision 1

6. Scaling results

Command	Result
`kubectl scale --replicas=6`	6/6 ready, 3 new pods ready within the rollout wait
`kubectl scale --replicas=2`	2/2, the newest pods removed first
`kubectl scale --replicas=3`	back to 3/3

Scaling changes only the replica count, so pods keep the same ReplicaSet hash and no new revision is recorded.

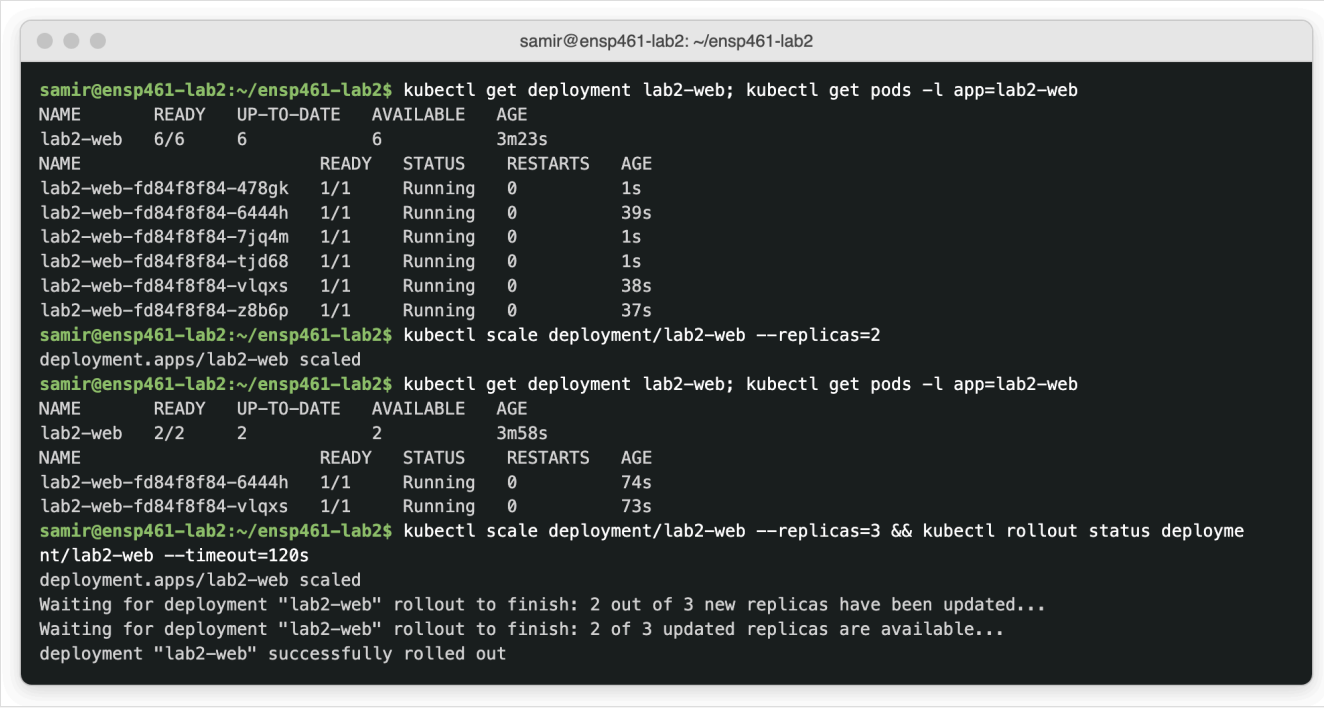


Fig: Scaling out and in

7. Continuous deployment

Build	Trigger	Version	Result
#1	SCM change		ABORTED: Jenkins restarted during checkout
#2	SCM change	2.1.2	SUCCESS, 176 s (first build: base image and pip download)
#3	SCM change (`/health` uptime commit)	2.1.3	SUCCESS, 27 s
#4	SCM change (manifest render fix)	2.1.4	FAILURE: smoke test hit a draining old pod; rolled back automatically
#5	SCM change (smoke test waits for drain)	2.1.5	SUCCESS, 30 s

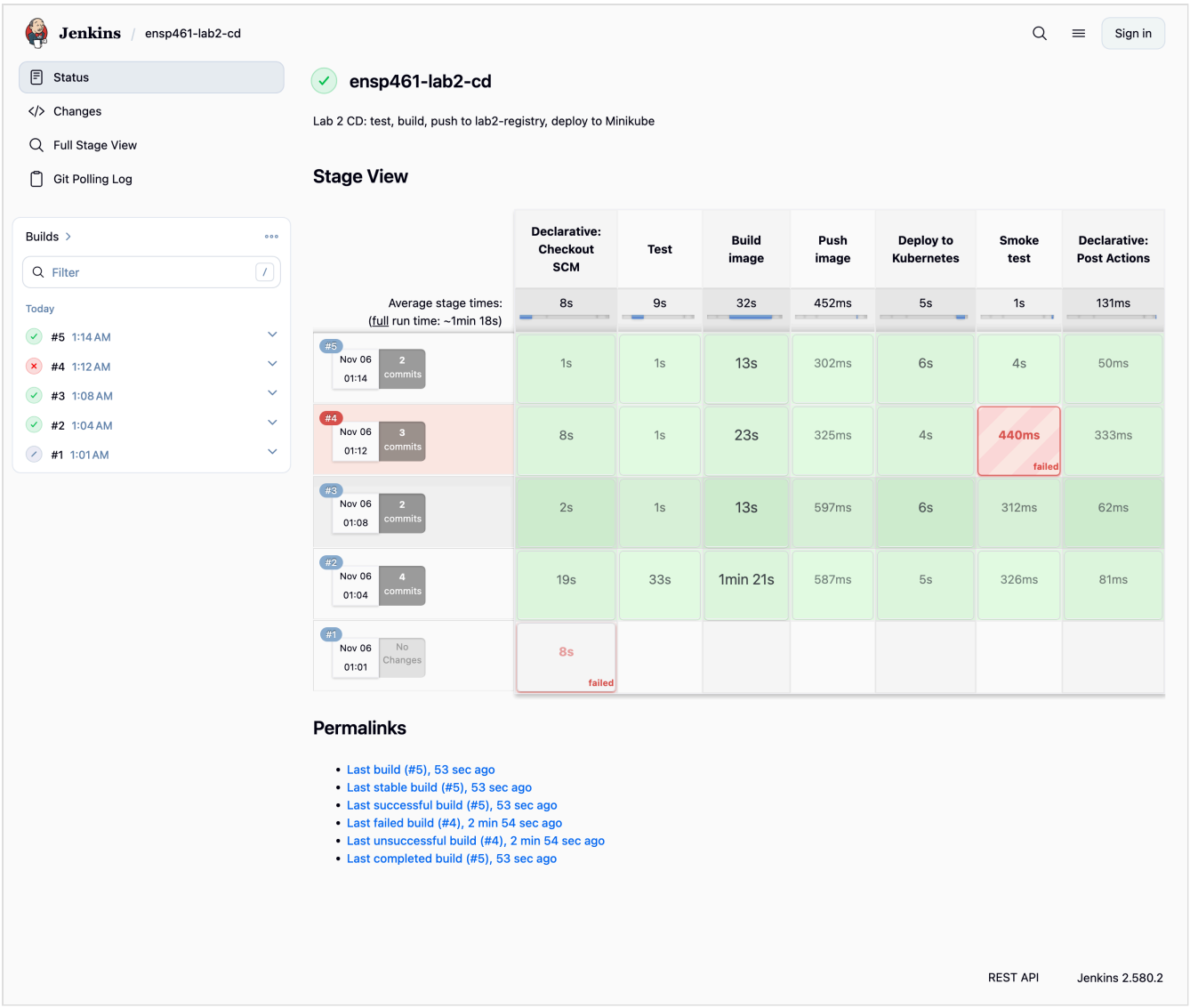


Fig: Jenkins stage view

8. Observations

- Layer order matters more than layer count: dependency installs belong before the frequently changing source.
- Containers must not depend on their writable layer for anything that must survive; volumes are the persistence boundary.
- Kubernetes readiness and rolling-update settings are necessary but not sufficient for zero downtime; graceful draining closed the gap.
- A failed release with `maxUnavailable: 0` degrades into a stalled rollout instead of an outage, which makes rollback calm.
- In CD, applying a manifest with a stale image tag and then patching it causes an extra rollout per deploy; render the final manifest once.
- A finished rollout does not mean old pods are gone; a smoke test must wait for the drain, and the automatic rollback proved itself when it did not.
- A self-hosted registry kept the lab independent of Docker Hub limits and credentials, and is the same pattern as a private registry in production.

9. Conclusion

The service runs as a non-root, health-checked container, as a Compose stack with persistent state, and as a three-replica Kubernetes Deployment that updates, rolls back and scales without failed requests. Jenkins deploys every pushed commit to the cluster and verifies the exact version is live. The full record of commands, logs, screenshots and fixes is in the repository and in the Q8 report.